
MOBILE DEEP LEARNING INFERENCE FRAMEWORK FOR ANDROID-BASED SHORT-CIRCUIT TRACE CLASSIFICATION

Supervisor: Prof. Junho Bang

Presenter: Hadi Nazari

IT Applied System Engineering

2026



전북대학교
JEONBUK NATIONAL UNIVERSITY

TABLE OF CONTENTS

Introduction

Objective

Proposed System

Dataset & Preprocessing

Android Implementation

Model Conversion (TFLite)

Results

Application Interface

Advantages

Conclusion



전북대학교
JEONBUK NATIONAL UNIVERSITY

INTRODUCTION

01

Electrical fires
often caused
by short
circuits

02

- Need to distinguish PSCT vs SSCT

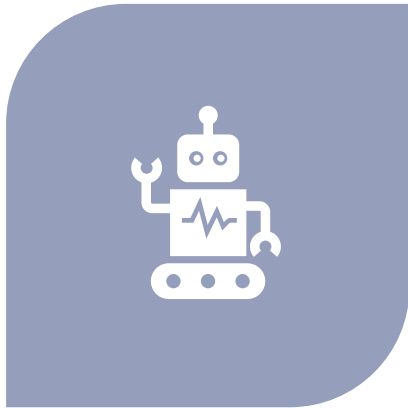
03

- Traditional methods are manual and subjective

04

- AI enables automated classification

OBJECTIVE



- DEVELOP ANDROID-BASED CLASSIFICATION SYSTEM



- USE LIGHTWEIGHT CNN MODELS

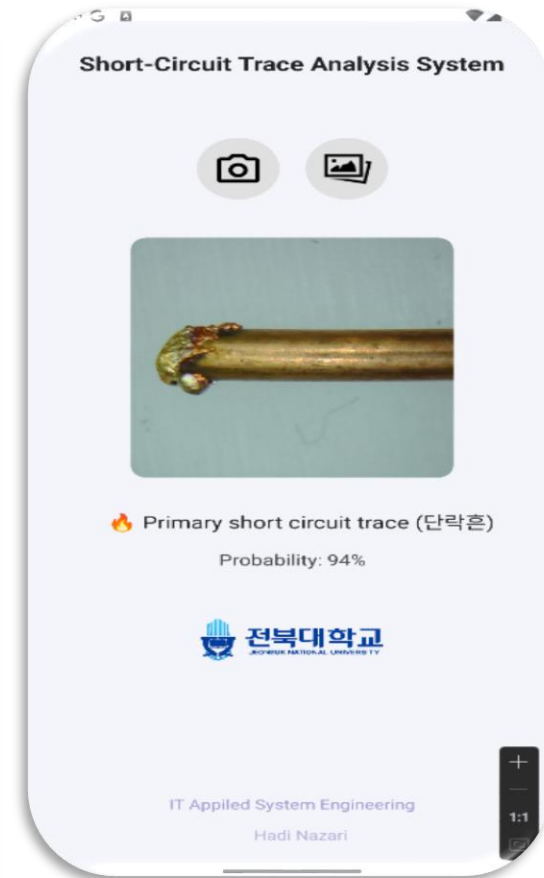
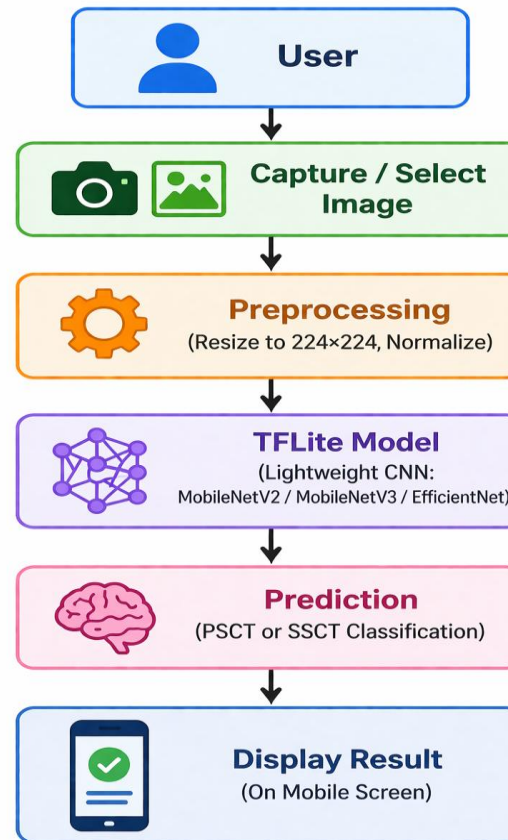
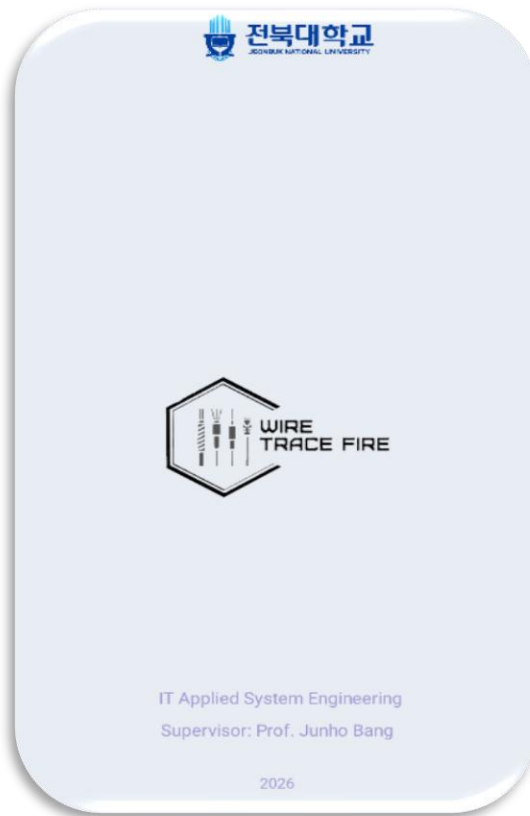


- ENABLE REAL-TIME MOBILE INFERENCE

PROPOSED FRAMEWORK FOR ELECTRIC FIRE SHORT-CIRCUIT TRACE CLASSIFICATION

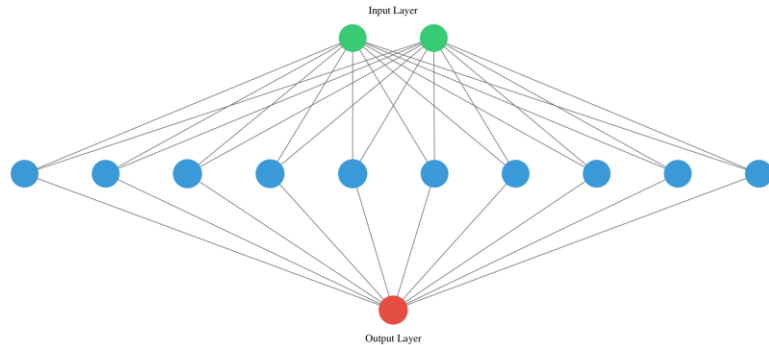


ANDROID SYSTEM OVERVIEW



ANDROID IMPLEMENTATION

- The mobile application was developed using:
 - **Java**
 - **Android Studio**
- Model trained in Python (Anaconda, TensorFlow)



DATASET



Total images: 752



- Classes: primary short-circuit traces (PSCT), secondary short-circuit traces (SSCT)



- Split: 80% training, 20% testing



- Augmentation: rotation, zoom, flip, brightness


```
java
└─ com.kda.kabul_decore_app
    └─ MainActivity
        └─ MainActivity.java
            40 public class MainActivity extends AppCompatActivity {
            130     private void loadModel() {
            131         try {
            132             MappedByteBuffer modelBuffer = loadModelFile(modelPath: "model.tflite");
            133             tflite = new Interpreter(modelBuffer);
            134             Log.d(TAG, msg: "✅ Model loaded successfully");
            135         } catch (IOException e) {
            136             Log.e(TAG, msg: "❌ Error loading model: " + e.getMessage());
            137             tvResult.setText("⚠️ !");
            138         }
            139     }
            140
            141     1 usage
            142     private MappedByteBuffer loadModelFile(String modelPath) throws IOException {
            143         AssetFileDescriptor fileDescriptor = getAssets().openFd(modelPath);
            144         FileInputStream inputStream = new FileInputStream(fileDescriptor.getFileDescriptor());
            145         FileChannel fileChannel = inputStream.getChannel();
            146         long startOffset = fileDescriptor.getStartOffset();
            147         long declaredLength = fileDescriptor.getDeclaredLength();
            148         return fileChannel.map(FileChannel.MapMode.READ_ONLY, startOffset, declaredLength);
            149     }
            150
            151     1 usage
            152     private ByteBuffer convertBitmapToByteBuffer(Bitmap bitmap) {
            153         int inputSize = 224;
            154         Bitmap scaledBitmap = Bitmap.createScaledBitmap(bitmap, inputSize, inputSize, true);
            155         ByteBuffer byteBuffer = ByteBuffer.allocateDirect(capacity: 4 * inputSize * inputSize);
            156         byteBuffer.order(ByteOrder.nativeOrder());
            157
            158         int[] pixels = new int[inputSize * inputSize];
            159         scaledBitmap.getPixels(pixels, offset: 0, inputSize, x: 0, y: 0, inputSize, inputSize);
            160
            161         for (int pixel : pixels) {
            162             int r = (pixel >> 16) & 0xFF;
            163             int g = (pixel >> 8) & 0xFF;
            164             int b = pixel & 0xFF;
```

PREPROCESSING

- Input image resized to: 224×224 pixels
- Converted into ByteBuffer format
- Pixel normalization applied for model compatibility



TensorFlow

MODEL CONVERSION (TENSORFLOW LITE)

- After training, the best-performing model was converted to TensorFlow Lite (.tflite) format.
- This step enables:
 - Reduced model size
 - Faster inference
 - Compatibility with mobile devices

INFERENCE PROCESS



Input image converted into numerical tensor



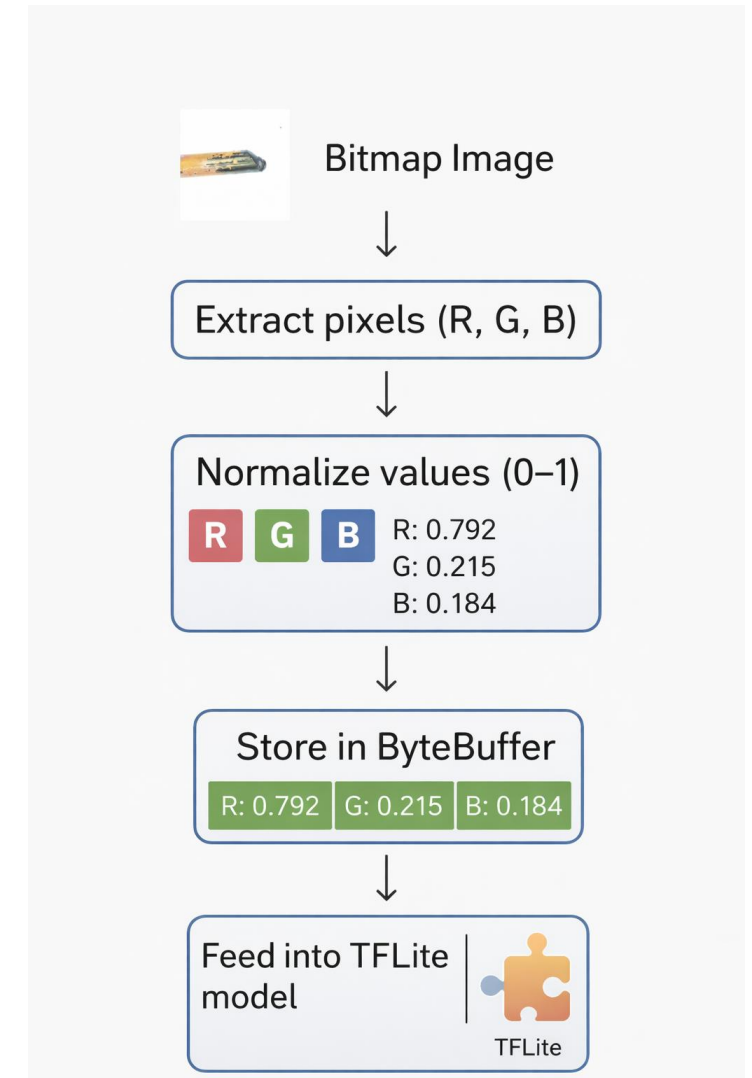
Model outputs a probability
value:

$< 0.5 \rightarrow \text{PSCT}$

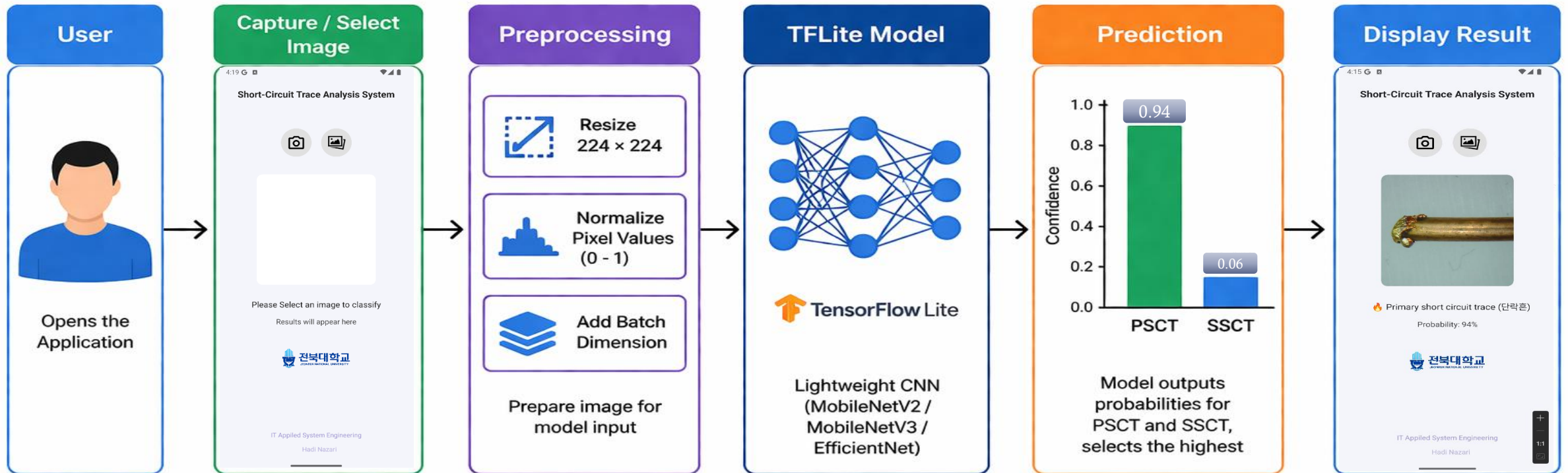
$\geq 0.5 \rightarrow \text{SSCT}$

BYTEBUFFER

- A **memory container for raw numerical data**
- TensorFlow Lite understands only numerical arrays (tensors).
- TensorFlow Lite requires input in this form because:
 - It is **fast**
 - It is **low-level (close to hardware)**
 - It avoids extra memory copying



SYSTEM OVERVIEW



RESULTS

EfficientNet: 96.05% accuracy

- MobileNetV3: stable performance
 - MobileNetV2: efficient and fast
-

APPLICATION INTERFACE

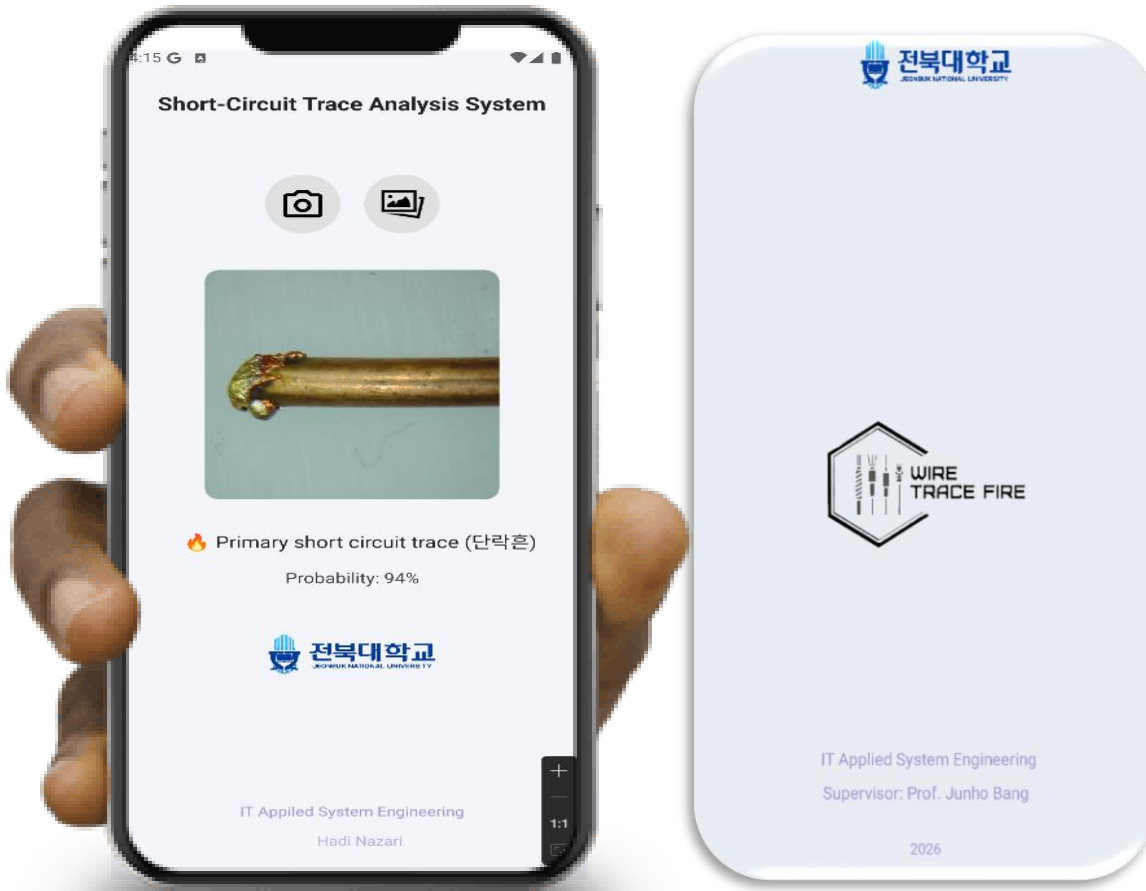


Image input (camera/gallery)

- Prediction result

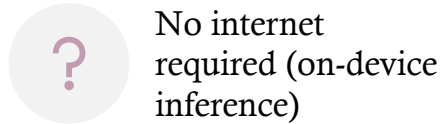
- Probability output

- User-friendly UI

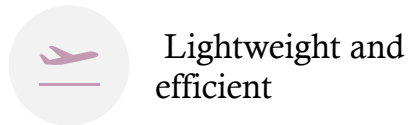
ADVANTAGES OF THE MOBILE IMPLEMENTATION



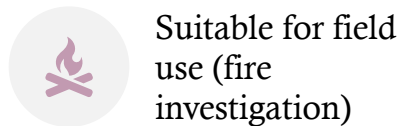
Real-time
classification



No internet
required (on-device
inference)



Lightweight and
efficient



Suitable for field
use (fire
investigation)



REAL TIME



SUMMARY



Model trained in Python (Anaconda, TensorFlow/Keras)



Converted to TensorFlow Lite format



Integrated into Android Studio (Java)



Supports camera & gallery input



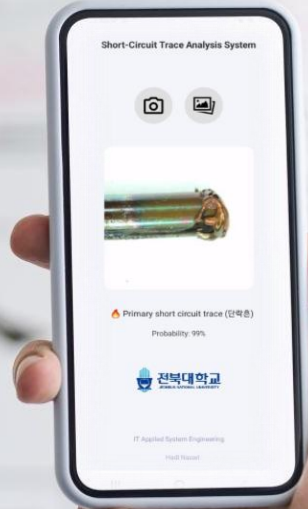
Performs real-time on-device classification



Displays predicted class and confidence

FUTURE WORK

- ❖ Expanding the dataset to improve generalization
- ❖ Enhancing accuracy using advanced models
- ❖ Improving the user interface with better graphics and usability
- ❖ Adding more features for practical and flexible use



THANK YOU



전북대학교
JEONBUK NATIONAL UNIVERSITY